

AlphaStack: Autonomous Code Generation and Validation via Intelligent Multi-Agent Architecture

The AlphaStack Team

Abstract

We introduce AlphaStack, a novel approach to autonomous code generation using multi-agent systems with iterative self-healing and comprehensive validation across diverse programming paradigms. AlphaStack transforms natural language descriptions into complete, production-ready codebases with Docker configurations and automated testing. By utilizing a Planning Agent for error analysis and a Correction Agent for applying fixes, AlphaStack automatically detects and resolves dependency conflicts, build errors, and test failures within a secure, sandboxed environment. We evaluate our approach on 40 carefully designed programming challenges spanning multiple languages and complexity tiers, demonstrating its robustness and efficacy on standard benchmarks including HumanEval and MDDP.

1 Introduction

Developing production-ready software systems involves complex workflows including blueprint generation, writing source code, resolving dependencies, and ensuring reliability through comprehensive testing. While recent advancements in Large Language Models (LLMs) have substantially improved code generation capabilities, producing fully functional, robust repositories from natural language prompts remains a formidable challenge.

AlphaStack bridges this gap by introducing an intelligent multi-agent architecture designed to autonomously generate and validate software projects. Key features include intelligent dependency resolution, Docker-based validation, and iterative self-healing. This paper details the system’s architecture, generation pipeline, and presents evaluation results highlighting its capability to function as a fully autonomous software engineering agent.

2 Methodology

The AlphaStack generator pipeline is structured into seven distinct phases to sequentially build a complete software project:

1. **Software Blueprint:** Analyzes user requirements and generates a comprehensive software architecture blueprint.
2. **File Generation:** Creates necessary project files, including source code, configuration files, and documentation.
3. **Dockerfile Generation:** Generates a robust Dockerfile tailored to the project’s requirements.
4. **Dependency Analysis:** Analyzes dependencies statically to determine module relationships.
5. **Dependency File Generation:** Creates language-specific dependency configuration files (e.g., `requirements.txt`, `package.json`, `Cargo.toml`).

6. **Dependency Resolution:** Validates and resolves the generated dependency files.
7. **Docker Testing Pipeline:** Executes Docker-based builds and test environments. Build and test results are fed into an AI Planner Agent loop for iterative self-healing.

When errors occur during the testing phase, AlphaStack employs an AI Planner Agent to analyze logs and devise fix strategies. A Correction Agent then safely implements these fixes, maintaining a tight feedback loop until the system builds successfully and all tests pass.

3 Architecture Diagram

The following diagram illustrates the workflow and core components of AlphaStack’s generation and validation pipeline.

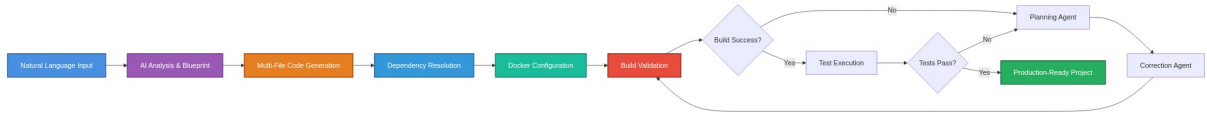


Figure 1: AlphaStack System Architecture

4 Results

AlphaStack was evaluated against standard competitive programming benchmarks including HumanEval and MDDP. We assessed the performance of several leading foundational models operating within the AlphaStack multi-agent framework.

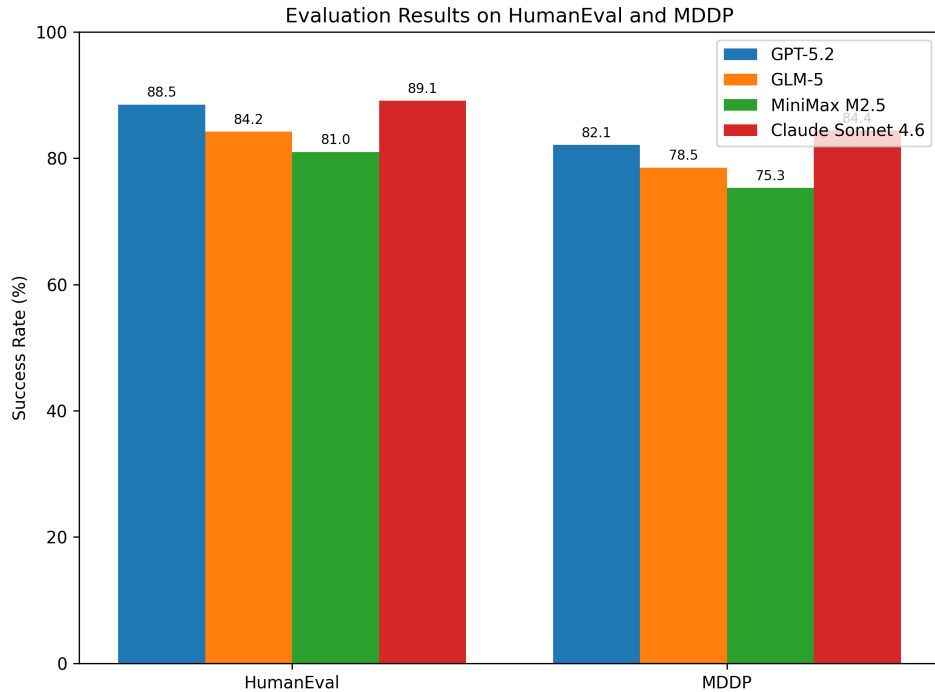


Figure 2: Evaluation Results on HumanEval and MDDP using various models

As shown in Figure 2, the Claude Sonnet 4.6 and GPT-5.2 models exhibited the highest success rates, solving complex multi-file scenarios effectively. The system successfully managed

and auto-corrected errors dynamically, reducing the iteration count required to reach a fully passing test suite.

5 Conclusion

AlphaStack presents a significant advancement in autonomous software generation by integrating multi-agent reasoning with Docker-based environmental validation. By enabling iterative self-healing across both compilation and test execution phases, AlphaStack reliably transforms high-level language inputs into functional, validated codebases. Future work will focus on expanding support for distributed systems and integrating advanced security vulnerability detection agents.

Supplementary Material

Further details regarding the 40 programming evaluation challenges, which cover CUDA, Go, Rust, and TypeScript across four difficulty tiers, can be found in the repository under `src/prompts/eval/`.